
HInT: Hypergraph Infusion at the Structural Layers

Improves Table Understanding

Wonjin Lee¹ Soomi Jeong¹ Kwang In Kim¹

Abstract

Decoder-only large language models (LLMs) struggle with table reasoning because tables must be serialized, which can obscure row- and column-level structure. Prior graph and hypergraph approaches encode structure with an external encoder, but their gains are often inconsistent under autoregressive decoding. We analyze how tabular structure is represented inside decoder-only LLMs and find that row and column relations are concentrated in a small subset of layers and attention heads. Based on this observation, we propose *HInT* (Hypergraph Infusion for Table reasoning), which injects hypergraph-derived structural features directly into the layers where these relations are concentrated. *HInT* constructs a table hypergraph over cells and headers, applies lightweight message passing, and fuses the resulting structural features with token hidden states through gated fusion. Experiments across diverse table reasoning tasks show consistent improvements over text-only baselines and prior (hyper)graph-based methods. Our code is available at <https://github.com/MLV-group/HInT.git>.

1. Introduction

Enabling language models to reason over tables is a key challenge across table reasoning tasks, including question answering (QA) (Pasupat & Liang, 2015; Nan et al., 2022; Cheng et al., 2022), table-text QA (Chen et al., 2020b; Nararatwong et al., 2022; Zhao et al., 2022), table-based fact verification (Chen et al., 2020a; Wu et al., 2023), and table-to-text generation (Parikh et al., 2020). A fundamental difficulty is the mismatch between the two-dimensional structure of tables and the one-dimensional token sequences

required by language models: linearizing tables row-wise or column-wise obscures explicit row and column relations and higher-order structure (Yang et al., 2022a; Long et al., 2025), often leading to brittle reasoning.

Earlier work, largely developed for sequence-to-sequence models, addressed this mismatch by introducing table-specific positional encodings, structured attention, or auxiliary objectives that preserve tabular structure (Liu et al., 2022; Jiang et al., 2022; Herzig et al., 2020; Yang et al., 2022b; Yin et al., 2020). Such techniques integrate naturally into models with a bidirectional encoding stage, including T5 (Raffel et al., 2020) and BART (Lewis et al., 2020).

More recently, decoder-only large language models (LLMs) have shown strong general-purpose reasoning, motivating their use for table reasoning (Sarkar & Lausen, 2023; Wu & Feng, 2024). However, purely autoregressive decoding removes the dedicated encoding stage and relies exclusively on causal attention, which makes it difficult to directly transfer structure-aware mechanisms developed for encoder-decoder architectures. As a result, most approaches for table reasoning with decoder-only LLMs focus on supervised fine-tuning over large corpora of tables (Sarkar & Lausen, 2023; Wu & Feng, 2024; Zhang et al., 2024b; 2025). While effective to some extent, these methods still process tables as linearized token sequences and thus inherit the same structural limitations. This raises a natural question: can explicitly encoding table structure outside the LLM reliably overcome the limitations of serialization for decoder-only models?

Inspired by VLMs that bridge modalities via dedicated encoders, recent table reasoning methods introduce (hyper)graph encoders to inject structural features before LLM decoding (Jin et al., 2025; Xu et al., 2025; Li et al., 2025). Graph-based representations are well suited for relational modeling, and hypergraphs are particularly natural for tables where cells participate in multiple semantic groupings (Yu et al., 2021; Chen et al., 2023).

However, in decoder-only LLMs, structural features produced by these (hyper)graph encoders are ultimately consumed through serialized causal attention. Tokens that are adjacent in a row or column may be far apart in the input sequence, limiting effective interaction between structurally

¹POSTECH, Korea. Correspondence to: Wonjin Lee <wonjin0403@postech.ac.kr>, Kwang In Kim <kimkin@postech.ac.kr>.

related elements during autoregressive reasoning (Kim et al., 2026; Pei et al., 2026; Zhao et al., 2025). In line with this limitation, we find that incorporating graph or hypergraph table encoders yields comparable, and in some cases lower, performance than text-only baselines when applied to decoder-only LLMs under a multi-step reasoning setting (see Figure 1). This issue is compounded by the fact that many encoder-based approaches emphasize short-answer supervision and provide limited support for explicit multi-step reasoning, which constrains how structural information can be used by decoder-only models (see Section D.4).

The bottleneck therefore lies not only in *extracting* table structure, but also in where and how it is *integrated* into autoregressive computation inside the LLM. To identify an effective integration point, we study how tabular structure is internally represented in decoder-only LLMs and find that row- and column-level relations are strongly expressed in a small subset of layers and attention heads. Based on this observation, we propose *HInT* (Hypergraph Infusion for Table reasoning), which injects hypergraph-derived structural information directly into these *structural layers*.

At each selected layer, *HInT* constructs a table hypergraph over cells and headers, performs hypergraph message passing, and fuses the resulting structural features back into token representations through lightweight gating, enabling structure-aware reasoning while preserving the LLM’s autoregressive computation.

Our experiments show that *HInT* improves performance across diverse table reasoning tasks, including table QA, fact verification, and table-to-text generation. We further show that *HInT* extends to more challenging multimodal and multi-table settings, where reasoning can require propagating structural constraints across multiple steps rather than only recovering local cell values.

2. Related Work

Text-based table reasoning. A common way to adapt language models to tabular inputs is to linearize tables into one-dimensional token sequences. This representation is widely used in recent LLM-based table reasoning systems, which rely on instruction tuning or supervised fine-tuning over large-scale table corpora to improve generalization and multi-step reasoning (Zhang et al., 2024b). Beyond question answering, TableLLM (Zhang et al., 2025) extends this paradigm toward spreadsheet-style table manipulation by training LLMs to perform a broader set of table operations with synthesized supervision signals.

Despite their empirical success, text-based approaches fundamentally treat tables as serialized text. As a result, row and column relations and higher-order structure are not explicitly represented and must be reconstructed

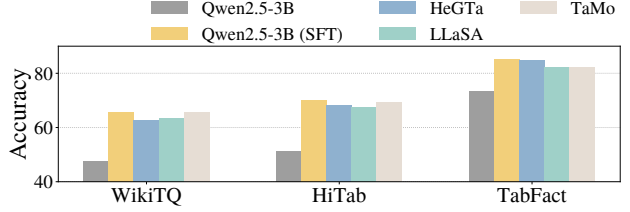


Figure 1. Comparison of text-only and graph-augmented LLMs on the WikiTQ, HiTab, and TabFact datasets. Qwen2.5-3B and Qwen2.5-3B (SFT; supervised fine-tuning) process tables purely as serialized text, whereas HeGTa (Jin et al., 2025) uses a GNN encoder, and LLaSA (Xu et al., 2025) and TaMo (Li et al., 2025) use HyperGNN encoders to model tabular structure. Despite explicitly modeling table structure, graph-augmented methods do not consistently outperform the text-only baselines.

implicitly through causal attention. In particular, row-wise serialization preserves local proximity within rows, while column-wise dependencies may span long token distances, making structure-sensitive reasoning brittle.

Graph- and hypergraph-based approaches. To explicitly represent tabular structure, many works model tables as graphs and encode them with GNNs, providing a more explicit representation of cell-level and layout-dependent relations than simple serialization (Du et al., 2021). Subsequent approaches introduce additional pretraining objectives, such as contrastive learning, to improve structure-aware representations in low-resource settings (Jin et al., 2023). Hypergraph formulations further generalize these models by representing rows, columns, and hierarchical headers as hyperedges, enabling higher-order and permutation-invariant relational modeling (Chen et al., 2023).

Building on this idea for LLM-based table reasoning, HeGTa (Jin et al., 2025) encodes tables as heterogeneous graphs, while LLaSA (Xu et al., 2025) and TaMo (Li et al., 2025) represent tables as hypergraphs and introduce modules to align structural and textual representations. While these methods provide explicit structural features, they primarily inject structure through external encoders before LLM decoding. Their gains can therefore remain inconsistent once inference proceeds through decoder-only autoregressive computation.

Internal structure integration. Rather than supplying structure through an external encoder, we integrate hypergraph-based structural information directly inside the LLM. Our key observation is that row- and column-level relations are concentrated in a small subset of layers and attention heads. *HInT* injects hypergraph-based structural features into these structurally aligned layers and fuses them with token representations, enabling structure-aware reasoning while preserving the autoregressive decoding process.

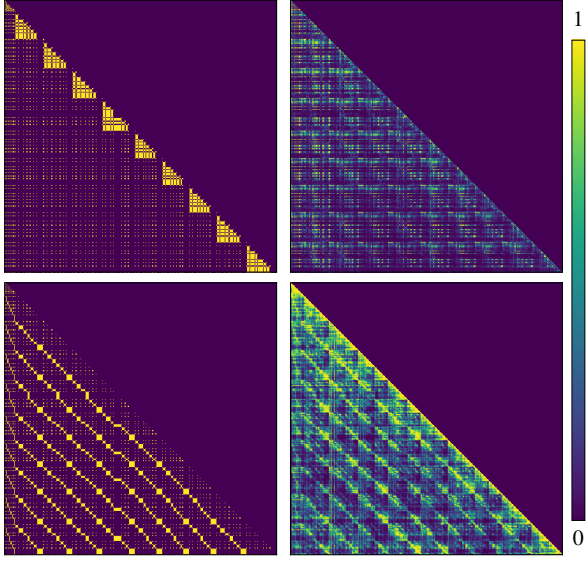


Figure 2. Left: row and column structural masks, M^R (top) and M^C (bottom). Right: attention maps from heads selected by our structural probing, showing row-wise and column-wise patterns that best match (i.e. top-1) the corresponding masks.

3. Hypergraph-based Structural Infusion

3.1. Problem Setup and Structural Bottleneck

Table reasoning aims to generate an answer A to a query Q grounded in one or more tables T . Each instance is represented as a triplet (T, Q, A) and modeled by the conditional distribution $p(A|T, Q)$.

Tables contain heterogeneous content such as numbers, text, and dates, and the semantics of Q and A depend on the task. For example, in table question answering (QA), Q is a natural language question and A is a span or a sentence grounded in the table. In fact verification, Q takes the form of a statement and A indicates whether it is supported by the table. In table-to-text generation, Q may be empty and A is a textual description of the table.

Decoder-only LLMs process tables through causal attention over serialized tokens. This enforces a sequential computation order and limits direct interaction between tokens that belong to the same row or column but are far apart in the linearized input. Consequently, structural relations central to table reasoning cannot be explicitly represented and must instead be encoded implicitly within attention patterns. This motivates a closer examination of where and how table structure is captured inside the LLM. Such implicit encoding is neither uniform across layers nor guaranteed to be preserved under downstream fine-tuning, which complicates attempts to inject structure at the input level. Understanding this internal heterogeneity is therefore critical for designing interventions that interact

Table 1. Table QA performance in Exact Match (EM, %) on WikiTQ under attention-head ablations. The top- K structurally aligned heads are removed and compared against removing an equal number of randomly selected heads.

Original	Ablate structural heads	Ablate random heads
47.51	12.73	32.29

constructively with autoregressive computation.

3.2. Structural Probing of LLM Attention

Due to the reliance on causal attention, any structural information required for table reasoning must be encoded within attention maps. Although attention operates over a serialized token sequence, certain attention heads may nonetheless capture interactions that correspond to row-wise and column-wise relationships among table elements. We probe whether structural information is preferentially associated with a subset of attention heads and layers.

Let the serialized table T and query Q be tokenized into N tokens. For each transformer layer $l \in \{1, \dots, L\}$ and attention head $h \in \{1, \dots, H\}$, let $\mathbf{A}_h^l \in \mathbb{R}^{N \times N}$ denote the attention matrix. We omit the explicit dependence on $X = (T, Q)$ for notational simplicity.

We define two binary structural masks, M^R and M^C , each of size $N \times N$, to encode idealized row-wise and column-wise interactions. For any pair of tokens (i, j) , $M^R(i, j) = 1$ if the two tokens belong to the same table row and 0 otherwise. The column mask M^C is defined analogously for table columns. These masks capture the intrinsic two-dimensional structure of tables independently of the one-dimensional token order induced by serialization.

Alignment between attention and table structure. We quantify how strongly attention heads encode tabular structure by measuring the alignment between their attention patterns and the structural masks M^R and M^C . Rather than matching each attention head to a mask independently, we model each structural mask as a sparse linear combination of attention maps across all heads and layers. Specifically, for the row mask M^R , we estimate combination coefficients $\mathbf{w}^R$ by solving the following LASSO problem:

$$\mathcal{C}(\mathbf{w}) = \left\| \mathbf{m}^R - \sum_{l,h} w_l^h \mathbf{a}_h^l \right\|_2^2 + \lambda \|\mathbf{w}\|_1, \quad (1)$$

where $\mathbf{a}_h^l$ and $\mathbf{m}^R$ are the vectorized attention map $\mathbf{A}_h^l$ and row mask M^R , respectively, for the input instance X , and $\mathbf{w} = \{w_l^h\}$ collects the coefficients for all heads. We set $\lambda = 1$ in all experiments. The ℓ_1 penalty promotes sparsity, so that only a small subset of heads receives nonzero weights, which we interpret as those whose attention patterns best align with row-wise structure. This probing does

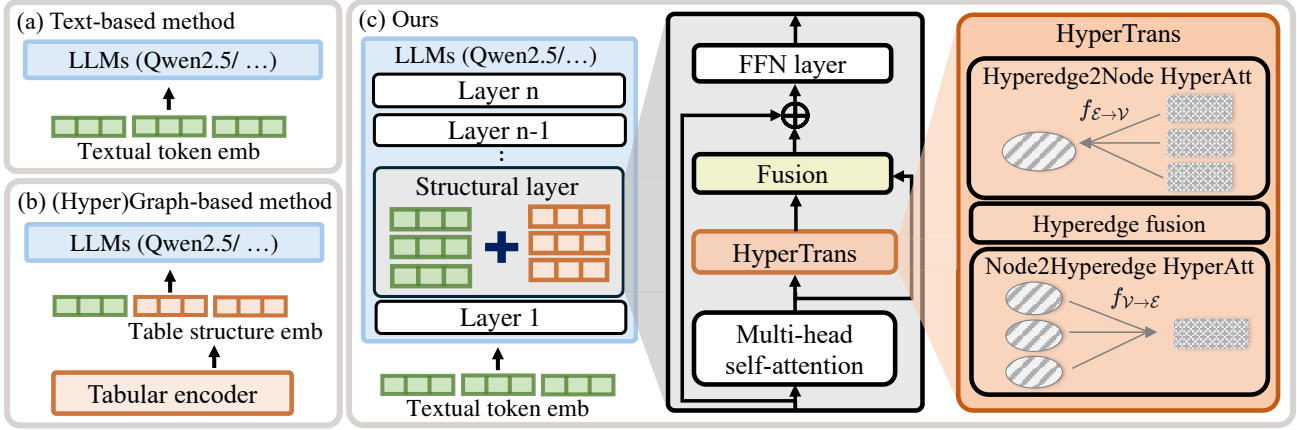


Figure 3. Comparison of structure integration strategies. (a) Text-based methods process tables after linearization into token sequences. (b) Graph- and hypergraph-based methods first construct a graph or hypergraph from the table, encode it with a dedicated encoder, and pass the resulting embeddings to the LLM. (c) *HInT* injects hypergraph-based structural features directly into selected structural layers of the LLM, where they are fused with hidden states to enable structure-aware reasoning.

not assume that structural information is localized to a single head or layer, allowing different instances to activate different subsets of structurally aligned components. We apply the same procedure to the column mask M^C to obtain w^C .

For each input instance, we rank heads by the magnitude of their coefficients. For visualization, Figure 2 shows the single highest-ranked (top-1) head for M^R and M^C , whose attention maps exhibit clear row-wise and column-wise patterns consistent with the corresponding structural masks, despite the serialized input format.

To test whether these structurally aligned heads are functionally relevant for table reasoning, we conduct a head removal study at inference time, removing the top $K = 40$ ranked heads and comparing against removal of K randomly selected heads. As shown in Table 1, ablating structural heads causes a substantially larger drop in table QA accuracy, indicating their critical functional role in structure-aware reasoning.

This suggests that tabular structure is not uniformly distributed across the LLM, but is concentrated in a small subset of computation pathways that actively support table reasoning. Rather than supplying structural representations to the model as external inputs, a more promising strategy is to intervene directly in these structurally active components and augment the computations that already encode row- and column-level relations. Since attention heads are parameterized within transformer layers, we intervene at the layer granularity where these structural heads concentrate.

Identification of structural layers. To identify where structural information is most consistently encoded, we aggregate the *structural heads* selected by the LASSO probe across input instances (WikiTQ; see Section 4). For

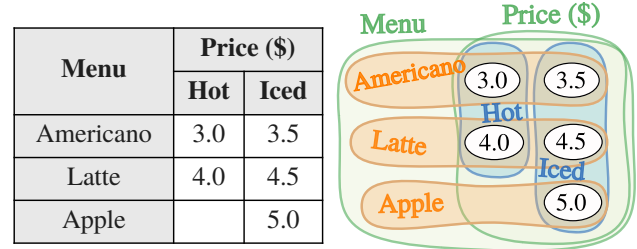


Figure 4. Example hypergraph construction from a table. Each table cell is represented as a node. Row-wise (orange) and column-wise (blue) hyperedges group cells sharing the same row or column, while hierarchical headers (green) induce higher-level hyperedges (e.g., *Price (\$)* spanning *Hot* and *Iced*).

each instance, the heads selected from both the row and column probes vote for their layers, yielding a frequency score that reflects how often each layer contributes to row-wise or column-wise structure.

This distribution is highly non-uniform (see Figure 6), and we designate the top-ranked layers as the *structural layers* (see Algorithm 2 and Section 3.5).

3.3. Hypergraph-Based Structural Feature Infusion

We now describe how explicit structural information is injected into the identified *structural layers*. Hypergraphs provide a natural representation for tables, as they capture higher-order relationships and support hierarchical and permutation-invariant structure. Following (Chen et al., 2023; Li et al., 2025), we adopt a hypergraph-based formulation to construct structure-aware table representations.

While the hypergraph encodes table structure explicitly, injecting it at every layer could interfere with the LLM’s

Algorithm 1 *HInT* Forward Pass

input Serialized tokens $\mathbf{x}_{1:N}$ derived from input $X = (T, Q)$; structural layer set $\mathcal{L}^*$

output Final hidden states $\mathbf{H}^L$

- 1: $\mathbf{H}^0 \leftarrow \text{Embed}(\mathbf{x}_{1:N})$
- 2: **for** $l = 1$ **to** L **do**
- 3: $\mathbf{U}^l \leftarrow \text{AddNorm}(\mathbf{H}^{(l-1)}, \text{SelfAttn}_l(\mathbf{H}^{(l-1)}))$
- 4: **if** $l \in \mathcal{L}^*$ **then**
- 5: Pool $\mathbf{U}^l$ to cell and header features (Eq. 2)
- 6: Apply hypergraph message passing (Eq. 3)
- 7: Fuse hypergraph features back to tokens (Eqs. 4–6)
- 8: $\tilde{\mathbf{U}}^l \leftarrow$ fused token representations
- 9: **else**
- 10: $\tilde{\mathbf{U}}^l \leftarrow \mathbf{U}^l$
- 11: **end if**
- 12: $\mathbf{V}^l \leftarrow \text{FFN}_l(\tilde{\mathbf{U}}^l)$ ▷ Feed-forward layer
- 13: $\mathbf{H}^l \leftarrow \text{AddNorm}(\tilde{\mathbf{U}}^l, \mathbf{V}^l)$
- 14: **end for**
- 15: **return** $\mathbf{H}^L$

language modeling dynamics. To preserve the model’s representational capacity while introducing explicit structural guidance, we inject hypergraph-based features only at the selected structural layers, leaving all other layers unchanged. An overview is shown in Figure 3.

Hypergraph construction. Given a table T , we construct a hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{1, \dots, N_c\}$ indexes table cells and $\mathcal{E} = \{e_1, \dots, e_{N_h}\}$ is a set of hyperedges, with each $e_v \subseteq \mathcal{V}$ grouping cells within the same row, column, or higher-level header (see Figure 4). Row-wise and column-wise hyperedges connect all cells within the same row or column, mirroring the relations captured by the structural masks. Hierarchical headers induce additional hyperedges that group rows or columns under higher-level categories (e.g., `Price ($)` spanning `Hot` and `Iced`).

For notational simplicity, we slightly overload notation and also use $\mathcal{V} = \{\mathbf{v}_u\}_{u=1}^{N_c}$ and $\mathcal{E} = \{\mathbf{e}_v\}_{v=1}^{N_h}$ to denote the associated node and hyperedge feature vectors, where $\mathbf{v}_u, \mathbf{e}_v \in \mathbb{R}^d$ and d is the hidden dimension of the LLM (e.g., 2048 for Qwen2.5-3B; see Section 4).

Let $\mathbf{u}_i \in \mathbb{R}^d$ denote the hidden representation of token i at the structural layer. Each node or hyperedge feature is initialized by average pooling the representations of tokens associated with the corresponding table cell or header:

$$\mathbf{v}_u = \frac{1}{|\mathcal{T}_u^\mathcal{V}|} \sum_{i \in \mathcal{T}_u^\mathcal{V}} \mathbf{u}_i, \quad \mathbf{e}_v = \frac{1}{|\mathcal{T}_v^\mathcal{E}|} \sum_{i \in \mathcal{T}_v^\mathcal{E}} \mathbf{u}_i, \quad (2)$$

where $\mathcal{T}_u^\mathcal{V}$ and $\mathcal{T}_v^\mathcal{E}$ denote the sets of tokens belonging to the u -th cell and the v -th header, respectively. For tables without explicit row headers, row indices are treated as

surrogate row hyperedges and included in $\mathcal{E}$.

Hypergraph message passing. We apply a HyperTrans module (Chen et al., 2023) to propagate structural information over the table hypergraph. HyperTrans jointly updates cell nodes and header hyperedges via cross-attention, enabling cells to exchange information through shared row, column, and hierarchical relations.

Let $\mathbf{V} \in \mathbb{R}^{N_c \times d}$ and $\mathbf{E} \in \mathbb{R}^{N_h \times d}$ denote the stacked cell and header features. The hypergraph layer produces structure-enhanced representations

$$(\tilde{\mathbf{V}}, \tilde{\mathbf{E}}) = \text{HyperTrans}(\mathbf{V}, \mathbf{E}), \quad (3)$$

where $\tilde{\mathbf{V}}$ and $\tilde{\mathbf{E}}$ preserve the original node and hyperedge cardinalities while encoding information aggregated along row, column, and header connections.

Token-level fusion. Each token i is associated with either a cell node or a header hyperedge. The corresponding structural feature is selected as

$$\tilde{\mathbf{g}}_i = \begin{cases} \tilde{\mathbf{v}}_u, & \text{if } i \in \mathcal{T}_u^\mathcal{V}, \\ \tilde{\mathbf{e}}_v, & \text{if } i \in \mathcal{T}_v^\mathcal{E}. \end{cases} \quad (4)$$

We then combine this structural feature with the token hidden state $\mathbf{u}_i$ via a sigmoid gate,

$$\alpha_i = \sigma(\mathbf{w}^\top [\mathbf{u}_i \| \tilde{\mathbf{g}}_i]), \quad \alpha_i \in (0, 1), \quad (5)$$

where $\|$ denotes concatenation, and obtain the fused token representation by a convex combination.

$$\tilde{\mathbf{u}}_i = (1 - \alpha_i) \mathbf{u}_i + \alpha_i \tilde{\mathbf{g}}_i. \quad (6)$$

This gated fusion injects structural cues in a token-adaptive manner, enabling row- and column-aware reasoning while preserving the LLM’s original language representations.

3.4. Training

HInT contains three trainable components: the backbone LLM, the HyperTrans hypergraph module, and the gating vector $\mathbf{w}$ in Eq. 5. The backbone LLM is initialized with Qwen2.5-3B-Instruct (Yang et al., 2024), while the gating vector $\mathbf{w}$ is initialized to all ones. The HyperTrans module is pretrained on tables from the WikiTQ training split using a self-supervised corruption detection objective over table headers and cells (Iida et al., 2021), without access to query or answer information.¹

All components are then jointly optimized end-to-end using *reasoning-augmented* supervision, in which the model is

¹Implementation by the HyperTrans authors: <https://github.com/aws-labs/hypergraph-tabular-lm/tree/main>.

trained to generate brief intermediate reasoning steps before producing the final answer. We adopt chain-of-thought style reasoning annotations provided in (Lei et al., 2025). This training setup differs from that used in prior graph- and hypergraph-based table reasoning methods, which are typically supervised only with short answers (Jin et al., 2025; Xu et al., 2025; Li et al., 2025). To ensure a fair comparison, we train each baseline under two setups: short-answer supervision and reasoning-augmented supervision, and report, for each dataset, the better-performing setting.

Details of prompt design and reasoning-augmented data construction are provided in Section C.4.

3.5. HInT Forward Pass

Algorithm 1 summarizes how *HInT* is integrated into a standard decoder-only transformer.² At each layer $l \in \{1, \dots, L\}$, the model first computes the post-attention representation $\mathbf{U}^l$. If l belongs to the structural layer set $\mathcal{L}^*$ identified in Section 3.2, $\mathbf{U}^l$ is passed through the hypergraph module and token-level gated fusion to produce a structure-infused representation $\tilde{\mathbf{U}}^l$. For all other layers, we set $\tilde{\mathbf{U}}^l = \mathbf{U}^l$. The transformer then applies the feed-forward block to obtain the layer- l hidden states $\mathbf{H}^l$.

In our experiments, we use Qwen2.5-3B as the backbone LLM. For this model, we compute layer-wise frequency scores from the top-40 attention heads per instance for both row- and column-wise probes. Applying the mean-score threshold yields 12 and 13 candidate layers for the row- and column-wise probes, respectively. Accordingly, we set the structural-layer budget to 12 and select

$$\mathcal{L}^* = \{3, 4, 5, 6, 8, 9, 10, 12, 15, 17, 24, 30\}.$$

We study how performance varies with the number of selected layers in Section 4.3.2.

4. Experiments

4.1. Experimental Setup

Datasets and evaluation metrics. We evaluate *HInT* on a broad range of table reasoning problems, covering 1) table question answering (QA), 2) table-based fact verification, 3) table-to-text generation (Table2Text), and 4) table-text (multimodal) QA. For table QA, we use WikiTableQuestions (WikiTQ) (Pasupat & Liang, 2015) and HiTab (Cheng et al., 2022). WikiTQ is a standard table QA benchmark with predominantly flat headers, whereas HiTab includes hierarchical column headers and row headers, requiring models to handle more complex table structure.

For fact verification, we use TabFact (Chen et al., 2020a). Although its tables are often structurally simpler than

HiTab, each statement may contain multiple compositional claims that must be jointly verified against the table, making reliable structural grounding important.

For Table2Text, we use FeTaQA (Nan et al., 2022) and ToTTo (Parikh et al., 2020). These benchmarks require free-form generation conditioned on tables, with ToTTo emphasizing faithful, concise summaries from highlighted cells and FeTaQA focusing on question-driven generation.

To test transfer to more complex settings, we further evaluate our method on table-text multimodal QA and multi-table QA benchmarks. HybridQA (Chen et al., 2020b) is a large-scale multi-hop question answering dataset that requires joint reasoning over tabular data and linked textual evidence, where each question is paired with a table and a related text passage. Correct answers can only be obtained by aggregating information across both modalities. In contrast, FinQA (Chen et al., 2021) focuses on numerical reasoning over financial documents and consists of QA pairs grounded in financial reports, requiring models to retrieve relevant facts from heterogeneous sources and perform multi-step numerical reasoning. FEVEROUS (Aly et al., 2021) is a fact verification benchmark involving heterogeneous evidence from both text and tables, including tables with hierarchical headers and multiple tables.

We evaluate table QA and table-text QA using Exact Match (EM), which measures whether the predicted answer exactly matches the reference answer. Table-to-text generation is evaluated using BLEU (Papineni et al., 2002), which measures n-gram overlap between the generated text and the reference description. Fact verification is evaluated using binary classification accuracy.

Baselines. We compare *HInT* against a set of baselines that differ in how tabular structure is handled. First, we include text-based approaches that process tables as linearized token sequences. These include TableLlama (Zhang et al., 2024b) and TableLLM (Zhang et al., 2025), which are LLaMA-based models originally trained via supervised fine-tuning on large-scale tabular corpora spanning multiple table reasoning tasks.

Because these models use backbones that differ from ours, we also construct Qwen2.5-based baselines for controlled comparison. Qwen2.5-3B (ZS) performs zero-shot inference via prompting only, while Qwen2.5-3B (SFT) is fine-tuned separately on each dataset.

To disentangle structural modeling from model capacity, we further introduce a capacity-matched baseline, Qwen2.5-3.3B, obtained by adding three standard transformer layers to Qwen2.5-3B so that it has the same number of parameters as *HInT* (see Section 4.3.1).

We also compare against (hyper)graph-based methods that

²A detailed version is given in Algorithm 3.

Table 2. Results on eight table-reasoning benchmarks across four task categories. Qwen2.5-3B (ZS) uses zero-shot prompting, while Qwen2.5-3B (SFT) is supervised fine-tuned on each dataset. For methods fine-tuned in our experiments, we report mean performance over multiple random seeds; the complete results with standard deviations are provided in Table 8. The best and second-best results for each benchmark are highlighted in **bold** and underlined, respectively.

Task type	Table QA		Fact verification		Table-text QA		Table2Text	
Dataset	WikiTQ	HiTab	TabFact	FEVEROUS	HybridQA	FinQA	FeTaQA	ToTTo
Text-based methods								
TableLlama	29.83	64.55	82.52	59.39	11.02	4.53	33.17	7.38
TableLLM	50.87	36.36	55.87	37.96	30.14	23.10	18.07	4.35
Qwen2.5-3B (ZS)	47.51	51.20	73.40	57.82	30.67	45.77	19.98	12.90
Qwen2.5-3B (SFT)	65.44	<u>70.12</u>	<u>85.25</u>	76.22	<u>39.92</u>	54.34	26.11	18.62
(Hyper)graph-based methods								
HeGTa	62.52	68.09	84.75	75.52	28.07	16.33	13.06	7.60
LLaSA	63.49	67.32	82.19	66.36	39.52	<u>55.30</u>	25.41	16.82
TaMo	<u>65.52</u>	69.28	82.32	<u>76.60</u>	39.35	54.12	26.21	<u>18.67</u>
<i>HInT (Ours)</i>	67.39	71.75	86.71	76.97	40.99	57.25	<u>26.58</u>	18.85
Metric	EM (% , $\uparrow$)		Accuracy (% , $\uparrow$)		EM (% , $\uparrow$)		BLEU (% , $\uparrow$)	

explicitly encode table structure, including HeGTa (Jin et al., 2025), TaMo (Li et al., 2025), and LLaSA (Xu et al., 2025). HeGTa employs a heterogeneous graph neural network to encode tables, while TaMo and LLaSA use hypergraph encoders, with LLaSA additionally introducing a module to align structural and textual representations. For all baseline models, we apply the same supervision and training protocol as *HInT* to ensure a fair comparison.

4.2. Main Results

HInT yields consistent gains over the backbone LLM (Qwen2.5-3B) across all benchmarks. As shown in Table 2, these gains appear not only on datasets with hierarchical or nested headers (e.g., HiTab and ToTTo), but also on benchmarks with relatively simple table layouts, suggesting broad robustness across table formats.

In contrast, existing (hyper)graph-based methods do not reliably outperform text-based baselines. Across benchmarks, their gains are inconsistent, and in several cases they match or underperform the SFT text-only baseline despite explicitly encoding table structure. This holds for HeGTa, LLaSA, and TaMo, whose performance varies widely across datasets without a clear or consistent advantage over text-only models. This aligns with our diagnosis that structure must be integrated into the autoregressive computation itself.

HInT addresses this limitation by injecting structural information directly into the LLM’s internally identified structural layers, augmenting computations that already encode row- and column-level relations and yielding

stronger gains across tasks.

One exception is FeTaQA, where none of the structure-aware methods outperform TableLlama, a text-based model. FeTaQA involves relatively simple table layouts, and the cells relevant to each question are explicitly highlighted, reducing the need for recovering long-range row/column relations. Importantly, *HInT* is the only approach that improves performance over the same backbone baseline (Qwen2.5-3B (SFT)), indicating that injecting structural information can still be beneficial even in such settings. Meanwhile, although ToTTo also provides highlighted cells, its tables are typically much longer, making long-range reasoning across distant rows and columns more important; in this regime, *HInT* continues to yield clear gains.

HInT also performs strongly on table-text multimodal benchmarks. On HybridQA, where answers require multi-hop reasoning over both tables and linked textual passages, *HInT* achieves the largest improvements, demonstrating that internal structure-aware representations facilitate joint reasoning over structured and unstructured evidence.

FEVEROUS presents particularly challenging settings involving multiple tables with hierarchical headers and interleaved text. In this scenario, *HInT* is the only method that consistently outperforms both text-based and (hyper)graph-based baselines, showing that the proposed approach generalizes effectively from single-table to multi-table reasoning.

Qualitative example. Figure 5 shows a representative WikiTQ instance requiring multi-step reasoning over row-

Table:	Qwen2.5-3B (SFT)’s response:																																																																																										
<table><tr><th>Rank</th><th>Player</th><th>County</th><th>Tally</th><th>Total</th><th>Opposition</th></tr><tr><td>1</td><td>Francis Forde</td><td>Galway</td><td>2-8</td><td>14</td><td>Roscommon</td></tr><tr><td>2</td><td>Niall English</td><td>Carlow</td><td>1-9</td><td>12</td><td>Westmeath</td></tr><tr><td>3</td><td>Kevin Broderick</td><td>Galway</td><td>3-1</td><td>10</td><td>New York</td></tr><tr><td>3</td><td>Gary Kirby</td><td>Limerick</td><td>1-7</td><td>10</td><td>Tipperary</td></tr><tr><td>3</td><td>Gary Kirby</td><td>Limerick</td><td>0-10</td><td>10</td><td>Tipperary</td></tr><tr><td>6</td><td>Seán McLoughlin</td><td>Westmeath</td><td>1-6</td><td>9</td><td>Carlow</td></tr><tr><td>6</td><td>David Martin</td><td>Meath</td><td>1-6</td><td>9</td><td>Offaly</td></tr><tr><td>6</td><td>Gary Kirby</td><td>Limerick</td><td>0-9</td><td>9</td><td>Antrim</td></tr><tr><td>9</td><td>John Byrne</td><td>Carlow</td><td>2-2</td><td>8</td><td>Westmeath</td></tr><tr><td>9</td><td>John Troy</td><td>Offaly</td><td>2-2</td><td>8</td><td>Laois</td></tr><tr><td>9</td><td>John Leahy</td><td>Tipperary</td><td>2-2</td><td>8</td><td>Kerry</td></tr><tr><td>9</td><td>Tom Dempsey</td><td>Wexford</td><td>1-5</td><td>8</td><td>Offaly</td></tr><tr><td>9</td><td>Paul Flynn</td><td>Waterford</td><td>1-5</td><td>8</td><td>Tipperary</td></tr><tr><td>9</td><td>Francis Forde</td><td>Galway</td><td>1-5</td><td>8</td><td>New York</td></tr></table>	Rank	Player	County	Tally	Total	Opposition	1	Francis Forde	Galway	2-8	14	Roscommon	2	Niall English	Carlow	1-9	12	Westmeath	3	Kevin Broderick	Galway	3-1	10	New York	3	Gary Kirby	Limerick	1-7	10	Tipperary	3	Gary Kirby	Limerick	0-10	10	Tipperary	6	Seán McLoughlin	Westmeath	1-6	9	Carlow	6	David Martin	Meath	1-6	9	Offaly	6	Gary Kirby	Limerick	0-9	9	Antrim	9	John Byrne	Carlow	2-2	8	Westmeath	9	John Troy	Offaly	2-2	8	Laois	9	John Leahy	Tipperary	2-2	8	Kerry	9	Tom Dempsey	Wexford	1-5	8	Offaly	9	Paul Flynn	Waterford	1-5	8	Tipperary	9	Francis Forde	Galway	1-5	8	New York	...Looking at the table, I'll scan the Player column for David Martin. Here he is: Rank 6, , Player David Martin, County Meath, Tally 1-6. So his tally is 1-6. Now I need to check the other players to see who else has 1-6 in the Tally column. Let me go through each row. Rank 1 is Francis Forde with 2-8. Not the same. ... Then Rank 3 has three players:...Next, Rank 6: David Martin is one of them, already checked. Then Rank 9 has several players. ... The other players in Rank 6 are David Martin and the others at rank 9... So no matches there. HiInT's response: ...Looking at the table, David Martin is listed under Rank 6. His row shows a Tally of 1-6. ...Now I need to check if any other player has the same Tally entry. Let me go through each row. Starting from the top: 1. Francis Forde: 2-8. Not the same. 2. Niall English: 1-9. No. 3. Kevin Broderick: 3-1. Nope. ...Next, looking at Rank 6 again. David Martin's entry: 1-6. Now check others in Rank 6. The other two players in Rank 6 are Seán McLoughlin and David Martin. Both have Tally 1-6. So they have the same tally. Wait, but the question says "other player" besides David Martin. So the answer would be Seán McLoughlin.
Rank	Player	County	Tally	Total	Opposition																																																																																						
1	Francis Forde	Galway	2-8	14	Roscommon																																																																																						
2	Niall English	Carlow	1-9	12	Westmeath																																																																																						
3	Kevin Broderick	Galway	3-1	10	New York																																																																																						
3	Gary Kirby	Limerick	1-7	10	Tipperary																																																																																						
3	Gary Kirby	Limerick	0-10	10	Tipperary																																																																																						
6	Seán McLoughlin	Westmeath	1-6	9	Carlow																																																																																						
6	David Martin	Meath	1-6	9	Offaly																																																																																						
6	Gary Kirby	Limerick	0-9	9	Antrim																																																																																						
9	John Byrne	Carlow	2-2	8	Westmeath																																																																																						
9	John Troy	Offaly	2-2	8	Laois																																																																																						
9	John Leahy	Tipperary	2-2	8	Kerry																																																																																						
9	Tom Dempsey	Wexford	1-5	8	Offaly																																																																																						
9	Paul Flynn	Waterford	1-5	8	Tipperary																																																																																						
9	Francis Forde	Galway	1-5	8	New York																																																																																						
Question: what other player had the same tally as david martin ?																																																																																											

Figure 5. Reasoning comparison on a WikiTQ test example. Ground-truth, intermediate attribute, and predictions are highlighted in green, orange, and blue, respectively. Top-right: Qwen2.5-3B (SFT) finds the correct **tally** but fails to propagate the row-wise constraint, yielding an incorrect answer. Bottom-right: *HInT* preserves the row-level dependency and answers correctly. For readability, only partial table content and abbreviated reasoning traces are shown. Full outputs are provided in Section D.10.

Table 3. Table QA performance on WikiTQ under capacity-matched and injection-location ablations. Qwen2.5-3.3B matches the parameter count of *HInT*. *HInT (Final)* injects structure into the identified structural layers, *HInT (Random)* into randomly selected layers, and *HInT (Full)* into all layers.

	# parameters	(EM, %)
Qwen2.5 (SFT)	3.0B	65.44
	3.3B	64.43
<i>HInT (Random)</i>	3.3B	65.71
<i>HInT (Full)</i>	3.7B	66.37
<i>HInT (Final)</i>	3.3B	67.39

wise relations. The question first identifies an intermediate attribute (the match **tally 1-6** for **David Martin**) and then requires retrieving another player with the same tally. The text-based baseline recovers the intermediate value but fails to propagate this constraint across rows, whereas *HInT* maintains the row-level dependency and correctly retrieves **Seán McLoughlin**, as reflected in the highlighted cells.

4.3. Additional Results and Ablations

4.3.1. EQUAL MODEL CAPACITY

HInT introduces additional parameters through the HyperTrans module and the gating mechanism. To verify that the observed improvements are not merely due to increased model size, we construct a capacity-matched baseline by appending three standard transformer layers to Qwen2.5-3B, yielding a 3.3B-parameter model.

As shown in Table 3, simply increasing model depth de-

grades performance, indicating that the gains come from targeted structural injection rather than additional parameters.

Table 4. Table QA performance on HiTab as the number of selected structural layers (SL) varies.

# SL	0	4	8	12	16	20
(EM, %)	70.10	70.25	71.15	71.84	71.78	71.21

4.3.2. STRUCTURAL INFORMATION INJECTION

Injection location: structural vs. random layers. As shown in Table 1, only a small subset of layers plays a critical role in encoding table structure. We therefore examine whether hypergraph-based structural features are most effective when injected into these identified structural layers, rather than into arbitrary layers of the model.

To this end, we construct a variant that injects the same structural information into randomly selected layers (*HInT (Random)* in Table 3). This variant slightly outperforms the text-only baseline, suggesting that structural cues are generally beneficial for table reasoning. However, it underperforms injection into the identified structural layers, indicating that the effectiveness of structural information depends strongly on where it is introduced within the network. This difference is consistent with the non-uniform distribution of structurally aligned heads observed in our probing analysis (Figure 6).

Number of structural layers. As shown in Table 4, performance improves as highly ranked structural layers are added, but starts to degrade when too many layers are injected. This trend is consistent with the random-layer

Table 5. Table QA performance on WikiTQ using Llama-3.1-8B-Instruct as the backbone LLM.

Model	(EM, %)
TableLlama	29.83
TableLLM (Llama3.1-8B)	50.87
Llama3.1-8B (SFT)	77.34
TaMo (Llama3.1-8B)	77.97
<i>HInT</i> (Llama3.1-8B)	80.47

injection result in Table 3, where untargeted injection is less effective than injection into the identified structural layers.

4.3.3. GENERALIZATION

TO A DIFFERENT BACKBONE FAMILY

To examine whether our findings extend beyond Qwen2.5-3B, we evaluate *HInT* using Llama-3.1-8B-Instruct, a larger model from a different backbone family (Table 5). Following the same probing procedure as in Section 3.2, we select 10 structural layers from its 32 transformer layers:

$$\mathcal{L}^* = \{0, 1, 2, 3, 4, 5, 6, 19, 23, 25\}.$$

HInT improves over the Llama-family text-based and SFT baselines, as well as the encoder-based TaMo variant. The layer-frequency distribution for Llama-3.1-8B-Instruct remains sparse, similar to Qwen2.5-3B (Figure 6).

4.3.4. PARAMETER-EFFICIENT FINE-TUNING

The experiments above use full fine-tuning to evaluate structural infusion under a strong task-adaptation setting. To further assess whether the proposed approach remains effective when only a small number of backbone parameters are updated, we also evaluate *HInT* under LoRA-based fine-tuning (LoRA FT), where the backbone is adapted through low-rank updates while the structural modules are trained for the target task (Table 6).

As shown in Table 6, LoRA-based *HInT* performs below its full-FT counterpart, but improves over the ZS and LoRA-FT SFT baselines. It also achieves higher accuracy than the LoRA-FT variants of LLaSA and TaMo. Even with low-rank backbone updates, structural infusion at the identified layers continues to provide gains over both text-only and structure-aware baselines. We further analyze the effect of fine-tuning on the backbone LLM’s general language capability in Section D.3.

5. Conclusions

We study how to incorporate tabular structure into decoder-only LLMs for table reasoning. Prior graph- and hypergraph-based methods introduce explicit structural features via

Table 6. Table QA performance on WikiTQ under LoRA-based fine-tuning (LoRA FT) and full fine-tuning (Full FT). All methods use Qwen2.5-3B as the backbone.

Method	Tuning setup	(EM, %)
ZS	N/A	47.51
SFT	LoRA FT	48.90
LLaSA	LoRA FT + table encoder	56.08
TaMo	LoRA FT + table encoder	57.90
<i>HInT</i>	LoRA FT + HyperTrans + gate	58.29
SFT	Full FT	65.44
<i>HInT</i>	Full FT + HyperTrans + gate	67.39

external encoders, but their gains are often inconsistent under autoregressive decoding. Our analysis shows that tabular structure is already encoded within decoder-only LLMs, yet concentrated in a small subset of layers and attention heads. Building on this observation, we introduce *HInT*, which injects hypergraph-derived structural features directly into the identified *structural layers* using lightweight message passing and token-level gated fusion. By augmenting computations that already capture row- and column-level relations while preserving standard autoregressive decoding, *HInT* yields consistent improvements over strong text-only and encoder-based baselines across diverse table reasoning tasks, without requiring a separate input-level structural encoder or modifying the underlying model architecture.

Although *HInT* is evaluated primarily on tabular reasoning tasks, the underlying principle of injecting structured relational information into internally aligned layers is not inherently limited to tables. Examining whether similar interventions can benefit other structured inputs, such as knowledge graphs or program representations, represents a natural extension of this work.

Acknowledgements

This work was supported by Samsung Electronics Co., Ltd. (IO240508-09825-01) and by the Korea government (MSIT) through the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant (No. RS-2019-II191906, Artificial Intelligence Graduate School Program (POSTECH)) and the National Research Foundation of Korea (NRF) grant (No. RS-2024-00337559).

Impact Statement

Since *HInT* builds on a pre-trained decoder-only LLM to perform table reasoning, it may inherit and reflect biases, inaccuracies, or misconceptions present in the underlying model. In addition, because *HInT* injects structural information into the model’s internal representations

rather than altering the data itself, any biases present in the original table content or training distributions can be amplified through structure-aware reasoning.

While we do not foresee specific harmful use cases arising from the proposed method, structure-aware table reasoning systems may be used to support selective or misleading interpretations of tabular data, particularly in settings where tables are curated or presented with an agenda. As with other LLM-based reasoning systems, care should therefore be taken when deploying such models in high-stakes or adversarial contexts.

References

- Aly, R., Guo, Z., Schlichtkrull, M. S., Thorne, J., Vlachos, A., Christodoulopoulos, C., Cocarascu, O., and Mittal, A. The fact extraction and VERification over unstructured and structured information (FEVEROUS) shared task. In *NeurIPS*, 2021.
- Chen, P., Sarkar, S., Lausen, L., Srinivasan, B., Zha, S., Huang, R., and Karypis, G. HyTrel: Hypergraph-enhanced tabular data representation learning. In *NeurIPS*, 2023.
- Chen, W., Wang, H., Chen, J., Zhang, Y., Wang, H., Li, S., Zhou, X., and Wang, W. Y. TabFact: A large-scale dataset for table-based fact verification. In *ICLR*, 2020a.
- Chen, W., Zha, H., Chen, Z., Xiong, W., Wang, H., and Wang, W. Y. HybridQA: A dataset of multi-hop question answering over tabular and textual data. In *Findings of EMNLP*, 2020b.
- Chen, Z., Chen, W., Smiley, C., Shah, S., Borova, I., Langdon, D., Moussa, R., Beane, M., Huang, T.-H., Routledge, B. R., et al. FinQA: A dataset of numerical reasoning over financial data. In *EMNLP*, 2021.
- Cheng, Z., Dong, H., Wang, Z., Jia, R., Guo, J., Gao, Y., Han, S., Lou, J.-G., and Zhang, D. HiTab: A hierarchical table dataset for question answering and natural language generation. In *ACL*, 2022.
- Du, L., Gao, F., Chen, X., Jia, R., Wang, J., Zhang, J., Han, S., and Zhang, D. TabularNet: A neural network architecture for understanding semantic structures of tabular data. In *ACM SIGKDD*, 2021.
- Herzig, J., Nowak, P. K., Müller, T., Piccinno, F., and Eisenschlos, J. TaPas: Weakly supervised table parsing via pre-training. In *ACL*, 2020. doi: 10.18653/v1/2020.acl-main.398.
- Iida, H., Thai, D., Manjunatha, V., and Iyyer, M. TABBIE: Pretrained representations of tabular data. In *NAACL*, 2021.
- Jiang, Z., Mao, Y., He, P., Neubig, G., and Chen, W. OmniTab: Pretraining with natural and synthetic data for few-shot table-based question answering. In *NAACL*, 2022.
- Jin, R., Wang, J., Tan, W., Chen, Y., Qi, G., and Hao, W. TabPrompt: Graph-based pre-training and prompting for few-shot table understanding. In *EMNLP*, 2023.
- Jin, R., Li, Y., Qi, G., Hu, N., Li, Y.-F., Chen, J., Wang, J., Chen, Y., Min, D., and Bi, S. HeGTa: Leveraging heterogeneous graph-enhanced large language models for few-shot complex table understanding. In *AAAI*, 2025.
- Kim, D., Ranjan, V., Nagata, T., Dhua, A., et al. Rethinking visual information processing in multimodal LLMs. In *ICLR 2026 Workshop MM Intelligence*, 2026.
- Lei, F., Meng, J., Huang, Y., Chen, T., Zhang, Y., He, S., Zhao, J., and Liu, K. Reasoning-Table: Exploring reinforcement learning for table reasoning. *arXiv:2506.01710*, 2025.
- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., and Zettlemoyer, L. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *ACL*, 2020.
- Li, L., Ye, C., Ye, W., Sun, Y., Jiang, Z., Wang, H., Tian, J., Zhang, Y., Wang, N., Fu, X., et al. Table as a modality for large language models. In *NeurIPS*, 2025.
- Liu, Q., Chen, B., Guo, J., Ziyadi, M., Lin, Z., Chen, W., and Lou, J.-G. TaPEX: Table pre-training via learning a neural SQL executor. In *ICLR*, 2022.
- Long, L., Gu, X., Sun, X., Ye, W., Wang, H., Wu, S., Chen, G., and Zhao, J. Bridging the semantic gap between text and table: A case study on NL2SQL. In *ICLR*, 2025.
- Nan, L., Hsieh, C., Mao, Z., Lin, X. V., Verma, N., Zhang, R., Kryściński, W., Schoelkopf, H., Kong, R., Tang, X., Mutuma, M., Rosand, B., Trindade, I., Bandaru, R., Cunningham, J., Xiong, C., and Radev, D. FeTaQA: Free-form table question answering. In *Trans. ACL*, 2022.
- Nararatwong, R., Kertkeidkachorn, N., and Ichise, R. Enhancing financial table and text question answering with tabular graph and numerical reasoning. In *ACL-IJCNLP*, 2022.
- Papineni, K., Roukos, S., Ward, T., and Zhu, W.-J. BLEU: A method for automatic evaluation of machine translation. In *ACL*, 2002.
- Parikh, A., Wang, X., Gehrmann, S., Faruqui, M., Dhingra, B., Yang, D., and Das, D. ToTTo: A controlled table-to-text generation dataset. In *EMNLP*, 2020.

- Pasupat, P. and Liang, P. Compositional semantic parsing on semi-structured tables. In *ACL-IJCNLP*, 2015.
- Pei, X., Huang, T., Ma, Y., and Xu, C. Rethinking causal mask attention for vision-language inference. In *ICLR*, 2026.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 2020.
- Sarkar, S. and Lausen, L. Testing the limits of unified seq2seq LLM pretraining on diverse table data tasks. In *NeurIPS 2023 Second Table Representation Learning Workshop*, 2023.
- Wu, Z. and Feng, Y. ProTrix: Building models for planning and reasoning over tables with sentence context. In *Findings of EMNLP*, 2024.
- Wu, Z., Hu, N., and Feng, Y. Enhancing structured evidence extraction for fact verification. In *EMNLP*, 2023.
- Xu, Y., He, S., Chen, J., ZengXiangrong, Z., Wang, B., Liu, G., Zhao, J., and Liu, K. LLaSA: Large language and structured data assistant. In *NAACL*, 2025.
- Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report. *arXiv:2412.15115*, 2024.
- Yang, J., Gupta, A., Shyam‘Upadhyay, He, L., Goel, R., and Paul, S. TableFormer: Robust transformer modeling for table-text encoding. In *ACL*, 2022a.
- Yang, J., Gupta, A., Upadhyay, S., He, L., Goel, R., and Paul, S. TableFormer: Robust transformer modeling for table-text encoding. In *ACL*, 2022b. doi: 10.18653/v1/2022.acl-long.40.
- Yin, P., Neubig, G., Yih, W., and Riedel, S. TaBERT: Pretraining for joint understanding of textual and tabular data. In *ACL*, 2020. doi: 10.18653/v1/2020.acl-main.745.
- Yu, T., Wu, C.-S., Lin, X. V., Wang, B., Tan, Y. C., Yang, X., Radev, D., Socher, R., and Xiong, C. GraPPa: Grammar-augmented pre-training for table semantic parsing. In *ICLR*, 2021.
- Zhang, Q., Singh, C., Liu, L., Liu, X., Yu, B., Gao, J., and Zhao, T. Tell your model where to attend: Post-hoc attention steering for LLMs. In *ICLR*, 2024a.
- Zhang, T., Yue, X., Li, Y., and Sun, H. TableLlama: Towards open large generalist models for tables. In *NAACL*, 2024b.
- Zhang, X., Luo, S., Zhang, B., Ma, Z., Zhang, J., Li, Y., Li, G., Yao, Z., Xu, K., Zhou, J., et al. TableLLM: Enabling tabular data manipulation by llms in real office usage scenarios. In *Findings of ACL*, 2025.
- Zhang, Y., Qian, S., Peng, B., Liu, S., and Jia, J. Prompt highlighter: Interactive control for multi-modal LLMs. In *CVPR*, 2024c.
- Zhao, H., Si, S., Chen, L., Zhang, Y., Sun, M., Chang, B., and Zhang, M. Looking beyond text: Reducing language bias in large vision-language models via multimodal dual-attention and soft-image guidance. In *EMNLP*, 2025.
- Zhao, Y., Li, Y., Li, C., and Zhang, R. MultiHiertt: Numerical reasoning over multi hierarchical tabular and textual data. In *ACL*, 2022.

A. Structural Probing and Layer Identification

Algorithm 2 summarizes our procedure for identifying the structural layer set $\mathcal{L}^*$. We construct a probing dataset $\mathcal{D}$ by randomly sampling 3,000 instances from the WikiTQ training split. Using $\mathcal{D}$, we identify attention heads that most consistently encode the row mask $\mathbf{M}^R$ and the column mask $\mathbf{M}^C$ by solving the LASSO objective in Equation (1).

For each $X \in \mathcal{D}$, we select the top- K heads ($K=40$) under both probes. Each selected head casts a vote for its layer, producing layer-wise frequency scores C^R and C^C that measure how often each layer contributes to structural encoding. We rank layers separately under the two masks and form $\mathcal{L}^*$ of size S by alternating between the row and column rankings. If only one slot remains, we compare the best remaining row layer under C^R with the best remaining column layer under C^C , and select the one with the larger score.

Layer-frequency distribution. Applying the selection procedure in Algorithm 2 with a target size of $S=12$ yields the final structural layer set

$$\mathcal{L}^* = \{3,4,5,6,8,9,10,12,15,17,24,30\}.$$

As shown in Figure 6, the layer-wise frequency scores are highly non-uniform, with the highest scores concentrated in a small subset of transformer layers.

B. HInT Forward Pass (Detailed)

Algorithm 3 details the forward pass of *HInT* from serialized input tokens, including how hypergraph-based structural features are injected at the structural layers $\mathcal{L}^*$ selected by our probing procedure.

C. Experimental Details

C.1. Training Settings

All experiments are conducted on a server equipped with 4×NVIDIA H200 GPUs, 1.5TiB RAM, and an AMD EPYC 9654 CPU, running Ubuntu 22.04.5. We use PyTorch 2.7.1, vLLM 0.10.1, and HuggingFace Transformers 4.56.2 with CUDA 12.8. Models fine-tuned in our experiments are trained for five epochs using AdamW with an initial learning rate of 5×10^{-6} and a batch size of 16.

C.2. Datasets

In Table 7, we categorize all benchmarks by their table setting (single-table vs. multi-table). We further annotate whether each dataset contains hierarchical headers (in column and/or row dimensions) and whether it involves heterogeneous input modalities (i.e., table-only, text-only, or combined table-and-text inputs).

C.2.1. TABLEQA

WikiTQ (Pasupat & Liang, 2015) consists of tables collected from Wikipedia and requires diverse operations such as superlatives, aggregations, and comparisons. It contains 11,321 training, 2,831 validation, and 4,344 test examples. On average, the test tables include 6.3 columns and 25.8 rows, with some tables reaching up to 21 columns and 517 rows.

HiTab (Cheng et al., 2022) complements WikiTQ by introducing hierarchical tables with multi-level headers and nested structures, commonly found in real-world documents such as financial reports and government statistics. It comprises 7,417 training examples and 1,584 test examples across 540 unique tables. The tables have an average of 9.9 columns and 12.8 rows, with up to 22 columns and 62 rows. HiTab emphasizes numerical reasoning and interpretation of structurally complex tables.

C.2.2. TABLE FACT VERIFICATION

TabFact (Chen et al., 2020a) is collected from 16,000 Wikipedia tables and focuses on verifying natural-language claims given a single table. It includes heterogeneous input settings, where evidence may be table-only (28,760 examples), text-only (34,963 examples), or combined table-and-text (24,667 examples). The train/validation/test splits contain 92,283 / 12,792 / 12,779 unique sentences and 13,182 / 1,696 / 1,695 unique tables, respectively. On average, tables contain 14 rows and 5.4 columns.

FEVEROUS (Aly et al., 2021) is a Wikipedia-based fact verification benchmark that supports evidence from both tables and text, and often involves multiple tables per claim. It also includes tables with hierarchical headers. The train/validation/test splits contain 71,291 / 7,890 / 7,845 examples.

C.2.3. TABLE-TEXT QA

HybridQA (Chen et al., 2020b) is a large-scale QA benchmark requiring reasoning over heterogeneous evidence, where each question is aligned with a Wikipedia table and associated textual context. The dataset contains 62,682 / 3,466 / 3,463 train/validation/test examples. On average, tables contain 15.7 rows and 4.4 columns.

FinQA (Chen et al., 2021) is collected from financial documents and focuses on complex multi-step numerical reasoning over tables and accompanying text. The benchmark includes table-only, text-only, and combined (table+text) input settings. The dataset provides 6,251 training, 883 validation, and 1,147 test examples (75%/10%/15% split). The input tables contain 6.36 rows on average and 59.42 tokens on average.

Algorithm 2 Identifying Structural Layers via LASSO Probing

input Dataset $\mathcal{D}$;
 attention maps $\{\mathbf{A}_h^l(X)\}_{l,h}$ for each $X \in \mathcal{D}$;
 structural masks $\{\mathbf{M}^T(X)\}_{T \in \{\mathbf{R}, \mathbf{C}\}}$
 $\lambda = 1$; $K = 40$;
 number of layers $S = 12$

output Structural layer set $\mathcal{L}^*$

- 1: Initialize counters $F^{\mathbf{R}}(l, h) \leftarrow 0$, $F^{\mathbf{C}}(l, h) \leftarrow 0$
- 2: **for** each instance $X \in \mathcal{D}$ **do**
- 3: Vectorize maps: $\mathbf{a}_h^l(X) \leftarrow \text{vec}(\mathbf{A}_h^l(X))$
- 4: **for** type $T \in \{\mathbf{R}, \mathbf{C}\}$ **do**
- 5: Vectorize mask: $\mathbf{m}(X) \leftarrow \text{vec}(\mathbf{M}^T(X))$
- 6: Solve LASSO to obtain coefficients $\mathbf{w}(X) = \{w_i^h(X)\}$:

$$\mathbf{w}(X) = \underset{\mathbf{w}}{\text{argmin}} \|\mathbf{m}(X) - \sum_{l,h} w_i^h \mathbf{a}_h^l(X)\|_2^2 + \lambda \|\mathbf{w}\|_1$$
 $\triangleright$ Eq. 1
- 7: Select the top- K heads by coefficient magnitude: $\mathcal{H}(X) \leftarrow \text{TopK}(\{|w_i^h(X)|\}_{l,h})$
- 8: **for** each $(l, h) \in \mathcal{H}(X)$ **do**
- 9: $F^T(l, h) \leftarrow F^T(l, h) + 1$
- 10: **end for**
- 11: **end for**
- 12: **end for**
- 13: Compute layer scores: $C^T(l) \leftarrow \sum_h F^T(l, h)$ for $T \in \{\mathbf{R}, \mathbf{C}\}$
- 14: $\mathcal{L}^* \leftarrow \emptyset$
- 15: **while** $|\mathcal{L}^*| < S$ **do**
- 16: $\mathcal{U} \leftarrow \{1, \dots, L\} \setminus \mathcal{L}^*$
- 17: **if** $|\mathcal{L}^*| = S - 1$ **then**
- 18: $l_{\text{best}} \leftarrow \underset{l \in \mathcal{U}}{\text{argmax}} \{C^{\mathbf{R}}(l), C^{\mathbf{C}}(l)\}$
- 19: $\mathcal{L}^* \leftarrow \mathcal{L}^* \cup \{l_{\text{best}}\}$
- 20: **else**
- 21: $l_r \leftarrow \underset{l \in \mathcal{U}}{\text{argmax}} C^{\mathbf{R}}(l)$; $l_c \leftarrow \underset{l \in \mathcal{U}}{\text{argmax}} C^{\mathbf{C}}(l)$
- 22: $\mathcal{L}^* \leftarrow \mathcal{L}^* \cup \{l_r, l_c\}$
- 23: **end if**
- 24: **end while**
- 25: **return** $\mathcal{L}^*$

C.2.4. TABLE2TEXT

FeTaQA (Nan et al., 2022) is a Wikipedia-based Table2Text benchmark consisting of tuples {table, question, free-form answer, supporting cells}. The tables include nested headers, and the free-form answers contain 18.9 words on average. The dataset provides 7,326 training, 1,001 validation, and 2,003 test examples. Tables contain 13.8 rows and 5.9 columns on average.

ToTTo (Parikh et al., 2020) is a large-scale Wikipedia-based table-to-text dataset, where annotators highlight relevant table cells and write a natural-language description. It contains 120,761 training and 7,700 test examples. The tables exhibit nested header structures and have 32.7 rows and 206.6 cells on average.

C.3. Baseline Setup

For comparable baselines, including text-based methods and (hyper)graph-based methods, we reproduce the results for TableLlama (Zhang et al., 2024b), TableLLM (Zhang et al., 2025), LLaSA (Xu et al., 2025) and TaMo (Li et al., 2025) using the code released by the respective authors. Since LLaSA pretrains its table encoder on 25 million tables collected by TaBERT (Yin et al., 2020), we use the pretrained parameters provided by the authors.

HeGTa (Jin et al., 2025) pretrains a graph-based encoder to produce input features for LLMs. However, to the best of our knowledge, its official implementation, training data, and pretrained model weights have not been publicly released. We therefore report results from our own reimplementation. We pretrain the encoder on a dataset constructed following the original paper and fine-tune it on

Algorithm 3 *HInT* Forward Pass

input Serialized input tokens $\mathbf{x}_{1:N}$ (token sequence length N); selected structural layer index set $\mathcal{L}^* \subset \{1, \dots, L\}$;
output Final hidden states $\mathbf{H}^L$

- 1: $\mathbf{H}^{(0)} \leftarrow \text{Embed}(\mathbf{x}_{1:N})$
- 2: **for** $l = 1$ **to** L **do**
- 3: $\mathbf{U}^l \leftarrow \text{AddNorm}(\mathbf{H}^{(l-1)}, \text{SelfAttn}_l(\mathbf{H}^{(l-1)}))$
- 4: **if** $l \in \mathcal{L}^*$ **then**
- 5: Let $\mathbf{U}^l = \{\mathbf{u}_i\}_{i=1}^N$ with $\mathbf{u}_i \in \mathbb{R}^d$
- 6: **for** $u = 1$ **to** N_c **do**
- 7: $\mathbf{v}_u \leftarrow \frac{1}{|\mathcal{T}_u^v|} \sum_{i \in \mathcal{T}_u^v} \mathbf{u}_i$ ▷ Pooling to node features (Eq. 2)
- 8: **end for**
- 9: **for** $v = 1$ **to** N_h **do**
- 10: $\mathbf{e}_v \leftarrow \frac{1}{|\mathcal{T}_v^e|} \sum_{i \in \mathcal{T}_v^e} \mathbf{u}_i$ ▷ Pooling to hyperedge features (Eq. 2)
- 11: **end for**
- 12: $\mathbf{V} \leftarrow [\mathbf{v}_1; \dots; \mathbf{v}_{N_c}] \in \mathbb{R}^{N_c \times d}$; $\mathbf{E} \leftarrow [\mathbf{e}_1; \dots; \mathbf{e}_{N_h}] \in \mathbb{R}^{N_h \times d}$
- 13: $\tilde{\mathbf{V}}, \tilde{\mathbf{E}} \leftarrow \text{HyperTrans}(\mathbf{V}, \mathbf{E})$ ▷ Hypergraph message passing (Eq. 3)
- 14: **for** $i = 1$ **to** N **do**
- 15: $\tilde{\mathbf{g}}_i \leftarrow \tilde{\mathbf{v}}_u$ **if** $i \in \mathcal{T}_u^v$; $\tilde{\mathbf{g}}_i \leftarrow \tilde{\mathbf{e}}_v$ **if** $i \in \mathcal{T}_v^e$
- 16: $\alpha_i \leftarrow \sigma(\mathbf{w}^\top [\mathbf{u}_i \parallel \tilde{\mathbf{g}}_i])$ ▷ Eq. 5
- 17: $\tilde{\mathbf{u}}_i \leftarrow (1 - \alpha_i)\mathbf{u}_i + \alpha_i \tilde{\mathbf{g}}_i$ ▷ Token-level fusion (Eq. 6)
- 18: **end for**
- 19: $\tilde{\mathbf{U}}^l \leftarrow \{\tilde{\mathbf{u}}_i\}_{i=1}^N$
- 20: **else**
- 21: $\tilde{\mathbf{U}}^l \leftarrow \mathbf{U}^l$
- 22: **end if**
- 23: $\mathbf{V}^l \leftarrow \text{FFN}_l(\tilde{\mathbf{U}}^l)$ ▷ Feed-forward layer
- 24: $\mathbf{H}^l \leftarrow \text{AddNorm}(\tilde{\mathbf{U}}^l, \mathbf{V}^l)$
- 25: **end for**
- 26: **return** $\mathbf{H}^L$

downstream tasks using the resulting pretrained weights. For a fair comparison, we build the pretraining dataset using the WikiTQ training tables, since our HyperTrans weights are trained on the same split.

C.4. Prompt Structure and Reasoning-Augmented Training Data

The input prompts for table reasoning tasks consist of fixed instructions and instance-specific fields, as shown in Figures 7 to 10. The fields enclosed in $\{\}$ correspond to values that vary across instances including the input table and query. For heterogeneous settings, additional evidence such as associated text is also included. We use this unified prompt format for supervised fine-tuning across all table reasoning datasets.

Each prompt specifies both the reasoning and answer formats, requiring the model to generate a brief intermediate reasoning trace before producing the final answer in a structured form. For supervision, we use table reasoning datasets whose reasoning traces and final answers are generated by large language models under the same unified

prompt format (Lei et al., 2025). We directly use these generated outputs together with the predefined prompt templates during training.

D. Additional Results and Analyses

D.1. Complete Main Results

Table 8 provides the complete results corresponding to Table 2, including standard deviations over multiple random seeds for the methods fine-tuned in our experiments.

D.2. Inference Latency and Peak Memory

We measure inference latency and peak memory on a single NVIDIA H200 GPU (batch size 1) using the same Transformers implementation with SDPA attention.

As shown in Table 9 and Table 10, *HInT* achieves latency comparable to the SFT baseline while incurring a modest memory overhead (approximately 7.5%) across table sizes.

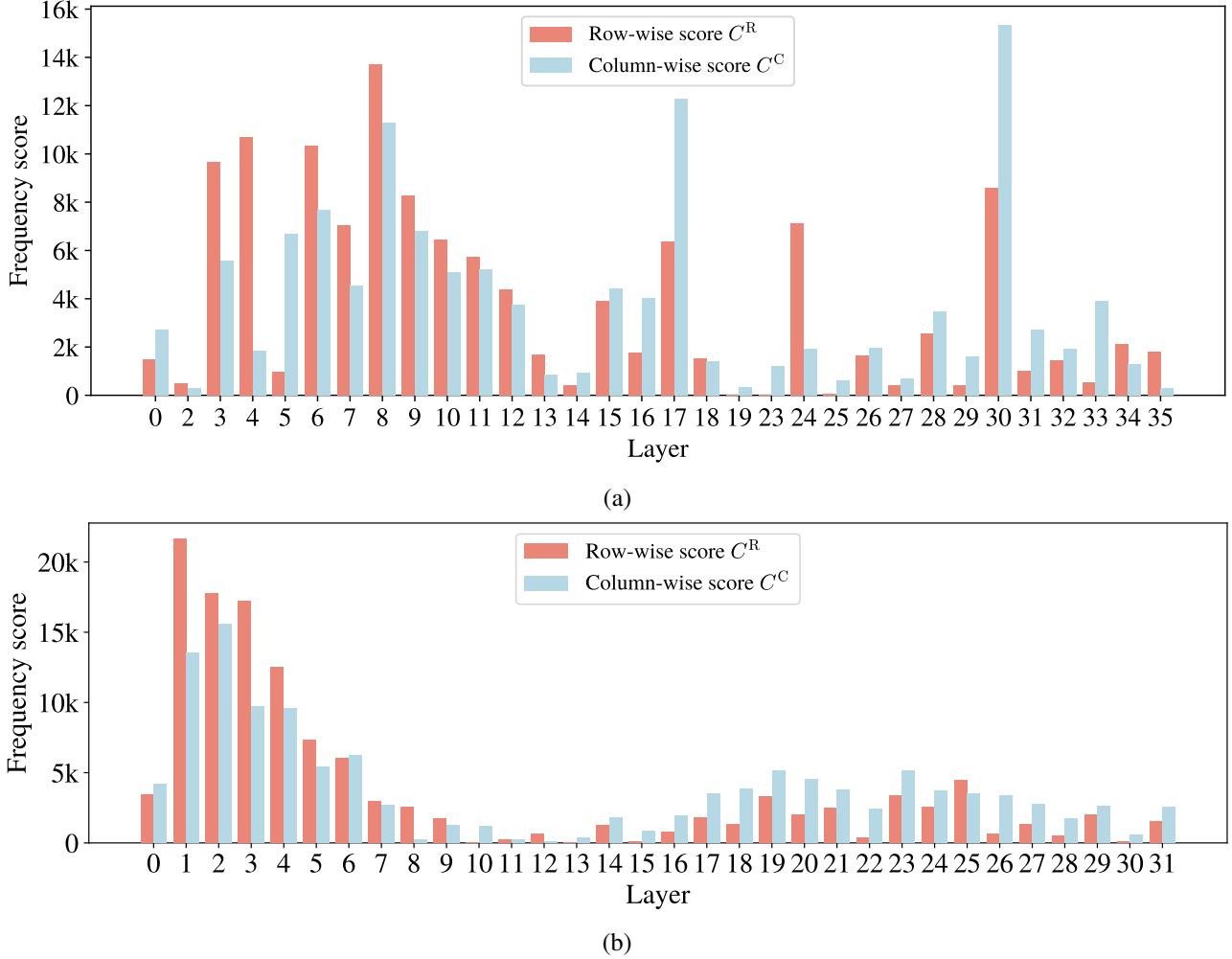


Figure 6. Layer-wise frequency scores of structurally aligned heads for (a) Qwen2.5-3B with 36 decoder layers and (b) Llama-3.1-8B with 32 decoder layers. Heads are identified by their alignment with the row mask M^R and column mask M^C . Both backbones show sparse, non-uniform distributions, indicating that row- and column-related structure is concentrated in a small subset of layers.

D.3. Impact of Fine-Tuning on General Language Capability

Optimizing the LLM backbone for a target task can introduce a trade-off between task-specific performance and general language ability. Since *HInT* fine-tunes the backbone together with the structural modules, we analyze this effect by reporting token perplexity on WikiText-2 for Qwen2.5-3B and *HInT* trained on WikiTQ, as shown in Table 11.

Under full fine-tuning, *HInT* shows an increase in perplexity compared with the original Qwen model. A similar increase is observed for Qwen2.5-3B with SFT, suggesting that *HInT* does not introduce additional degradation beyond standard supervised fine-tuning.

To further examine the impact on downstream behavior, we evaluate FEVEROUS, which includes text-only, table-only, and text+table subsets. As shown in Table 12, *HInT* results in only a marginal drop on the text-only subset (-0.26%),

while improving performance on the table-only ($+2.52\%$) and text+table ($+1.05\%$) subsets, with improvements primarily on tasks that require reasoning over table structure.

For LoRA-based tuning, token perplexity remains comparable to that of the original zero-shot LLM.

D.4. Short-Answer vs. Reasoning-Augmented Supervision

Figure 1 reports the performance of existing graph- and hypergraph-based table-encoding methods under a multi-step reasoning setting. In this setting, encoder-based approaches are generally comparable to, and in some cases lower than, text-only baselines.

Table 13 presents the corresponding results under short-answer supervision, which is the evaluation protocol originally adopted in prior work. Under this setting, graph- and hypergraph-based table-encoding methods show

Table 7. Summary of table reasoning benchmarks.

Task	Dataset	Table type	Hierarchical column header	Hierarchical row header	Heterogeneous (table + text)
Table QA	WikiTQ	Single	✗	✗	✗
	HiTab	Single	✓	✓	✗
Fact verification	TabFact	Single	✗	✗	✗
	FEVEROUS	Multi	✓	✓	✓
Table-text QA	HybridQA	Single	✗	✗	✓
	FinQA	Single	✗	✗	✓
Table2Text	FeTaQA	Single	✓	✓	✗
	ToTTo	Single	✓	✓	✓

WikiTQ prompt

Objective

You are provided with a table and a question related to the table. Your task is to answer the question step by step based on the table.

Table: {table}

Question: {query}

The answer should be short and simple. It can be a number, a word, or a phrase in the table, but not a full sentence. Your response should end with 'Answer: xxx' (answer to the question). Now, answer the question step by step:

HiTab prompt

You are capable of effectively identifying the hierarchical structure of the table. Based on the provided table and textual description, please provide the answer to the question. You should think step-by-step to reach the final answer.

Table: {table}

Question: {query}

Response

Your response should end with 'Answer: xxx' (answer to the question).

Figure 7. Example prompts for the table QA. Given a query Q and a table T , we form a text prompt in which variable fields are enclosed in {}.

improved performance over text-only baselines, although the overall accuracy remains lower than that obtained with multi-step reasoning. The effectiveness of table-encoding methods depends on the supervision and reasoning regime, and trends observed under short-answer supervision may not carry over to multi-step reasoning.

D.5. Comparison with a Simple Structural Bias

To assess whether a simpler structural bias is sufficient, we implement an attention-highlighting baseline that emphasizes row- and column-aligned tokens by directly modifying attention maps (Zhang et al., 2024a;c). As shown in Table 14, attention highlighting underperforms the SFT baseline and is substantially below *HInT*, suggesting that simple attention reweighting alone does not effectively capture table structure.

TabFact prompt

```
## Task:
You will predict the claim's label based on the given claim and table. There
are 2 possible labels: SUPPORTS, REFUTES. SUPPORTS means the claim is supported
by the table. REFUTES means the claim is refuted by the table. You should think
step-by-step to reach the final answer.

### Table: {table}

### Claim: {query}

### Response
Your response should end with 'Answer: xxx' (answer to the question).
```

FEVEROUS prompt

```
## Task:
You will predict the claim's label based on the given claim and table (text). There
are 3 possible labels: SUPPORTS, REFUTES, and NOT ENOUGH INFO. SUPPORTS means the
claim is supported by the table or text. REFUTES means the claim is refuted by
the table or text. NOT ENOUGH INFO means the table (text) does not provide enough
information to make the decision. You should think step-by-step to reach the final
answer.

## Table: {table}

## Text: {text}

## Claim: {query}

## Response
Your response should end with 'Answer: xxx' (answer to the question).
```

Figure 8. Example prompts for table fact verification. Given a claim (query) Q and a table T , we form a text prompt in which variable fields are enclosed in $\{\}$. For the heterogeneous dataset FEVEROUS (bottom), the input additionally includes textual evidence.

D.6. Transferability of Identified Structural Layers

In our method, structural layers are identified once on WikiTQ and then fixed across all benchmarks, without dataset-specific tuning. This raises a possible concern that selecting structural layers from a fixed dataset may introduce domain bias. However, as shown in Table 2, this single layer set consistently improves performance across diverse benchmarks, suggesting that the selected layers are not specific to WikiTQ.

To further examine transferability, we compare WikiTQ-based layer selection with random layer selection on HiTab. As shown in Table 17, WikiTQ-based selection substantially outperforms random selection. This comparison provides additional evidence that the identified layers capture structure-relevant computation beyond the source dataset. A more systematic evaluation under explicit domain shifts is left for future work.

D.7. Robustness to Serialization Format Changes

While our main experiments use a fixed table serialization format, the identified structural layers are intended to capture relationships inherent to tables rather than properties tied to a specific surface representation. To examine this, we evaluate *HInT* on a subset of WikiTQ using Markdown format at inference time, without retraining.

As shown in Table 16, the baseline model shows a slight performance drop when the input format is changed from the original serialization to Markdown. *HInT* also experiences a modest degradation, but it consistently outperforms the baseline under both formats.

D.8. Comparison with Frontier Zero-Shot Systems

We compare *HInT* (built on Qwen2.5-3B) against a frontier zero-shot (ZS) system (GPT-5.4) on HiTab and FEVEROUS (Table 15).

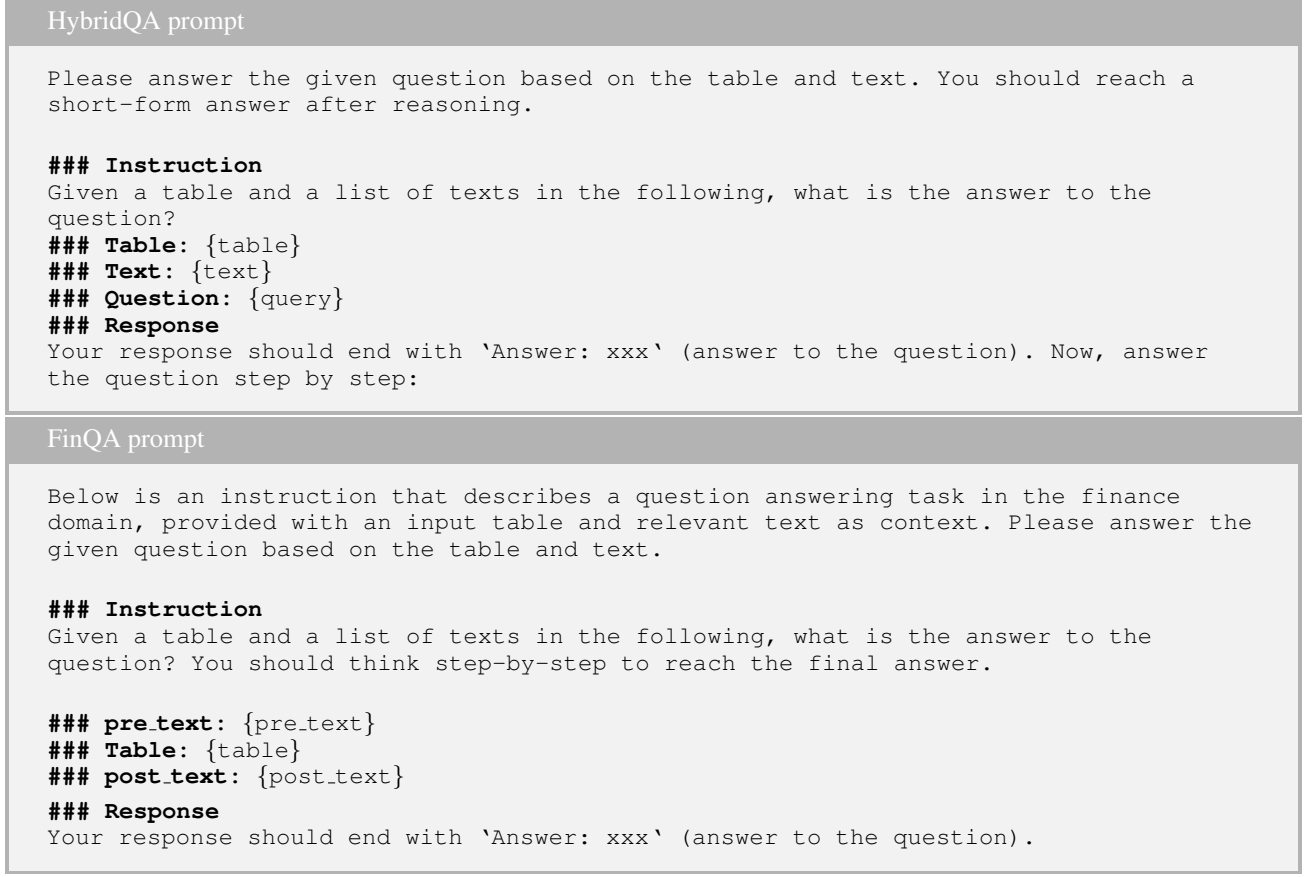


Figure 9. Example prompts for table-text QA. In addition to one or more tables T and the query Q , the prompt also includes textual evidence. Variable fields are enclosed in {}.

This is not a strictly direct comparison, since *HInT* is fine-tuned on a smaller open backbone whereas GPT-5.4 is evaluated in a zero-shot setting. On HiTab, which contains complex tables with nested headers, *HInT* achieves higher accuracy than GPT-5.4. On FEVEROUS, whose tables are relatively simpler, *HInT* performs somewhat below GPT-5.4 but remains competitive.

D.9. Visualization of Structural Attention Shifts

To further analyze the effect of structural fusion, we visualize attention maps before and after fusion. Figure 11 shows that, although raw attention maps appear visually similar, the difference maps reveal a redistribution toward row- and column-aligned regions matching the structural mask, indicating reinforced table-aware attention patterns.

D.10. Qualitative Examples

Unabridged reasoning traces for Figure 5 are shown in Figure 13, and the corresponding table and question are provided in Figure 12. As described in Section 4, this

example requires multi-step reasoning: the model must first identify David Martin’s match tally (1-6) and then retrieve another player with the same tally.

As shown at the top of Figure 13, the text-based baseline, Qwen2.5-3B (SFT), correctly finds that David Martin’s tally is 1-6, but fails to identify the other player with the same value. Although it performs step-by-step reasoning by checking ranked players and their tallies, it fails to retrieve Seán McLoughlin at rank 6, indicating that it does not capture the required row-wise dependency and therefore misses the correct entity.

In contrast, *HInT* correctly recognizes that both David Martin and Seán McLoughlin appear at rank 6 and verifies that they share the same tally 1-6. It therefore selects Seán McLoughlin instead of David Martin.

E. Limitations and Future Work

The current implementation relies on a fixed set of *structural layers* identified through offline probing on a specific backbone model. Although our ablation studies

HInT: Hypergraph Infusion at the Structural Layers Improves Table Understanding

Table 8. Results on eight table-reasoning benchmarks across four task categories. Qwen2.5-3B (ZS) uses zero-shot prompting, while Qwen2.5-3B (SFT) is supervised fine-tuned on each dataset. For methods fine-tuned in our experiments, we report mean $\pm$ standard deviation over multiple random seeds. The best and second-best results for each benchmark are highlighted in **bold** and underlined, respectively.

Task type	Table QA		Fact verification		Table-text QA		Table2Text	
Dataset	WikiTQ	HiTab	TabFact	FEVEROUS	HybridQA	FinQA	FeTaQA	ToTTo
Text-based methods								
TableLlama	29.83	64.55	82.52	59.39	11.02	4.53	33.17	7.38
TableLLM	50.87	36.36	55.87	37.96	30.14	23.10	18.07	4.35
Qwen2.5-3B (ZS)	47.51	51.20	73.40	57.82	30.67	45.77	19.98	12.90
Qwen2.5-3B (SFT)	65.44 $\pm$ 0.02	<u>70.12$\pm$0.32</u>	<u>85.25$\pm$0.36</u>	76.22 $\pm$ 0.31	<u>39.92$\pm$0.29</u>	54.34 $\pm$ 0.70	26.11 $\pm$ 0.14	18.62 $\pm$ 0.03
(Hyper)graph-based methods								
HeGTa	62.52 $\pm$ 1.83	68.09 $\pm$ 0.62	84.75 $\pm$ 0.53	75.52 $\pm$ 0.41	28.07 $\pm$ 2.58	16.33 $\pm$ 1.56	13.06 $\pm$ 1.79	7.60 $\pm$ 3.65
LLaSA	63.49 $\pm$ 1.48	67.32 $\pm$ 1.68	82.19 $\pm$ 2.82	66.36 $\pm$ 2.18	39.52 $\pm$ 0.53	<u>55.30$\pm$3.78</u>	25.41 $\pm$ 2.67	16.82 $\pm$ 1.59
TaMo	<u>65.52$\pm$0.19</u>	69.28 $\pm$ 0.71	82.32 $\pm$ 2.46	<u>76.60$\pm$0.20</u>	39.35 $\pm$ 0.04	54.12 $\pm$ 0.88	26.21 $\pm$ 0.09	<u>18.67$\pm$0.04</u>
<i>HInT (Ours)</i>	67.39$\pm$0.16	71.75$\pm$0.08	86.71$\pm$0.16	76.97$\pm$0.18	40.99$\pm$0.42	57.25$\pm$0.35	<u>26.58$\pm$0.31</u>	18.85$\pm$0.11
Metric	EM (% , $\uparrow$)		Accuracy (% , $\uparrow$)		EM (% , $\uparrow$)		BLEU (% , $\uparrow$)	

Table 9. Inference latency (ms) by table size (mean $\pm$ standard deviation).

# Cells	<i>HInT</i>	Qwen2.5-3B (SFT)
0–50	8,809 $\pm$ 16,033	8,204 $\pm$ 13,827
50–100	10,148 $\pm$ 16,274	10,176 $\pm$ 16,293
100–200	11,563 $\pm$ 15,927	12,326 $\pm$ 18,104
200–400	18,442 $\pm$ 25,808	17,984 $\pm$ 25,079
400+	22,112 $\pm$ 30,721	21,193 $\pm$ 30,369

Table 10. Peak memory (MB) by table size (mean $\pm$ standard deviation).

# Cells	<i>HInT</i>	Qwen2.5-3B (SFT)
0–50	6,421 $\pm$ 32	5,970 $\pm$ 28
50–100	6,446 $\pm$ 37	5,996 $\pm$ 36
100–200	6,498 $\pm$ 48	6,045 $\pm$ 48
200–400	6,637 $\pm$ 162	6,174 $\pm$ 154
400+	7,073 $\pm$ 442	6,581 $\pm$ 411

indicate that performance is relatively robust to moderate variations in the number of selected layers, extending this identification procedure to different model architectures, scales, or domains remains an open challenge. Developing mechanisms that adaptively identify structurally relevant layers, potentially in an online or data-dependent manner, is an important direction for further investigation.

Our hypergraph formulation focuses on row-wise, column-wise, and hierarchical header relations, which capture the dominant structural patterns in many existing table reasoning benchmarks. We do not explicitly model other

Table 11. Token perplexity on WikiText-2 under LoRA-based fine-tuning (LoRA FT) and full fine-tuning (Full FT). All methods use Qwen2.5-3B as the backbone.

Model	Tuning setup	Token perplexity ($\downarrow$)
ZS	N/A	7.40
SFT	LoRA FT	7.48
<i>HInT</i>	LoRA FT + HyperTrans + gate	7.42
SFT	Full FT	8.07
<i>HInT</i>	Full FT + HyperTrans + gate	7.96

Table 12. Accuracy (%) on FEVEROUS by evidence type.

Model	Text-only	Table-only	Text-table
Qwen2.5-3B (SFT)	78.19	74.05	74.47
<i>HInT</i>	77.93	76.57	75.52

forms of table semantics, such as numerical constraints, functional dependencies between cells, or relations spanning multiple tables. Extending the hypergraph construction to incorporate such signals is complementary to the proposed framework and may further enhance structure-aware reasoning in more complex settings.

Beyond the tabular setting, although *HInT* is evaluated primarily on table reasoning tasks, the underlying principle of injecting structured relational information into internally aligned layers is not inherently limited to tables. Examining whether similar interventions can benefit other forms of structured inputs, such as knowledge graphs or program representations, represents a natural extension of this work.

Table 13. Performance (EM, %) of text-only and graph-augmented LLMs on WikiTQ and TabFact under short-answer supervision. Qwen2.5-3B (SFT) processes tables as serialized text, while HeGTa (Jin et al., 2025), LLaSA (Xu et al., 2025), and TaMo (Li et al., 2025) use graph or hypergraph encoders to model tabular structure

Model	WikiTQ	TabFact
Qwen2.5-3B (SFT)	51.89	80.32
HeGTa	49.94	54.79
LLaSA	54.44	80.14
TaMo	52.95	79.58

Table 14. Table QA performance (EM, %) on WikiTQ for a simple attention-highlighting baseline that emphasizes row- and column-aligned tokens.

Model	EM (%)
Qwen2.5-3B (ZS)	47.51
Qwen2.5-3B (SFT)	65.44
Attention highlighting	62.78

Table 15. Performance (EM, %) on HiTab and FEVEROUS comparing *HInT* with a frontier zero-shot system, GPT-5.4. *HInT* and Qwen2.5-3B (SFT) are fine-tuned models, whereas GPT-5.4 is evaluated zero-shot using the prompt in Section C.4.

Model	HiTab	FEVEROUS
GPT-5.4 (ZS)	35.73	82.36
Qwen2.5-3B (SFT)	70.12	76.22
<i>HInT</i>	71.75	76.97

Table 16. Table QA performance (EM, %) on WikiTQ under different table serialization formats. Models are trained using the original serialization format and evaluated with Markdown serialization at inference time without retraining.

Model	Markdown	Original format
Qwen2.5-3B (SFT)	65.42	65.44
<i>HInT</i>	66.54	67.39

Table 17. Table QA performance (EM, %) on HiTab using randomly selected structural layers and structural layers identified from WikiTQ.

Randomly selected	WikiTQ-based selected
46.43	71.75

FeTaQA prompt

Task:

You are given a table in one of the following formats: html, plain text, or json. The table is accompanied by a context. The context is in the form of page title, table title, section title. The table has some highlighted cells. These cells are wrapped with `<hl>` and `</hl>` tags. Your task is to answer the question based on the table and the text content, and the answer should be a single sentence. The sentence must have the following characteristics:

- it must be able to answer what the question asks
- it can't contain lists
- it must be only a single sentence long
- it should focus on the content of the highlighted cells wrapped with `<hl>` and `</hl>` tags
- it must not explicitly mention that some cells are highlighted

TABLE: {table}

CONTEXT:

page title: {page_title}
section title: {section_title}

Question: {query}

Response

Your response should end with 'Answer: xxx' (the final sentence).

ToTTo prompt

Task:

You are given a table in one of the following formats: html, plain text, or json. The table is accompanied by a context. The context is in the form of page title, table title, section title. The table has some highlighted cells. These cells are wrapped with `<hl>` and `</hl>` tags. Your task is to answer the question based on the table and the text content, and the answer should be a single sentence. The sentence must have the following characteristics:

- it must be able to answer what the question asks
- it can't contain lists
- it must be only a single sentence long
- it should focus on the content of the highlighted cells wrapped with `<hl>` and `</hl>` tags
- it must not explicitly mention that some cells are highlighted

TABLE: {table}

CONTEXT:

page title: {page_title}
section title: {section_title}
section text: {section_text}

Question: {query}

Response

Your response should end with 'Answer: xxx' (the final sentence).

Figure 10. Example prompts for Table2Text generation. Given a query Q and a table T with contextual metadata, we form a text prompt that asks for a single-sentence description grounded in highlighted cells. The variable fields are enclosed in {}.

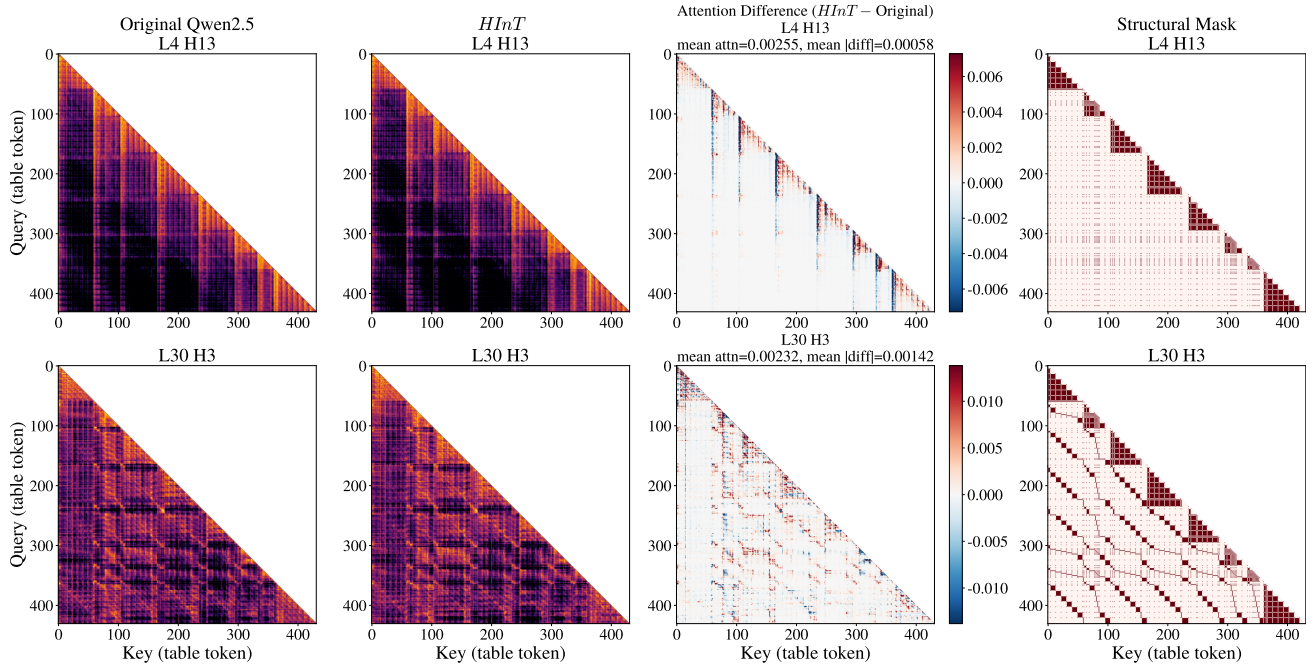


Figure 11. Example visualizations of structural attention heads. (First and second columns) attention maps of the original Qwen2.5 model and *HInT*, respectively; (Third column) their differences (*HInT*-original); (Fourth column) the structural masks.

Table:

	Rank	Player	County	Tally	Total	Opposition
row 1	1	Francis Forde	Galway	2-8	14	Roscommon
row 2	2	Niall English	Carlow	1-9	12	Westmeath
row 3	3	Kevin Broderick	Galway	3-1	10	New York
row 4	3	Gary Kirby	Limerick	1-7	10	Tipperary
row 5	3	Gary Kirby	Limerick	0-10	10	Tipperary
row 6	6	Seán McLoughlin	Westmeath	1-6	9	Carlow
row 7	6	David Martin	Meath	1-6	9	Offaly
row 8	6	Gary Kirby	Limerick	0-9	9	Antrim
row 9	9	John Byrne	Carlow	2-2	8	Westmeath
row 10	9	John Troy	Offaly	2-2	8	Laois
row 11	9	John Leahy	Tipperary	2-2	8	Kerry
row 12	9	Tom Dempsey	Wexford	1-5	8	Offaly
row 13	9	Paul Flynn	Waterford	1-5	8	Tipperary
row 14	9	Francis Forde	Galway	1-5	8	New York

Question:

what other player had the same tally as david martin?

Figure 12. Question and full table for the example in Figure 5.

Qwen2.5-3B (SFT)'s response:

<think>Okay, let's see. The question is asking which other player had the same tally as David Martin. First, I need to find David Martin's tally in the table.

Looking at the table, I'll scan the Player column for David Martin. Here he is: Rank 6, Player David Martin, County Meath, Tally 1-6, Total 9, Opposition Offaly. So his tally is 1-6.

Now I need to check the other players to see who else has 1-6 in the Tally column.

Let me go through each row. Rank 1 is Francis Forde with 2-8. Not the same.

Rank 2 is Niall English with 1-9. Then Rank 3 has three players: Kevin Broderick with 3-1, Gary Kirby with 1-7, and another Gary Kirby with 0-10. Wait, no, that's three entries for Gary Kirby here. But their Tally is 1-7, 0-10, and 0-10. So those aren't 1-6.

Next, Rank 6: David Martin is one of them, already checked. Then Rank 9 has several players. Let's check those. John Byrne has 2-2. John Troy: 2-2. John Leahy: 2-2. Tom Dempsey: 1-5. Paul Flynn: 1-5. Francis Forde again: 1-5.

So none of the Rank 9 players have a tally of 1-6. Wait, let me double-check. The only other player with 1-6 is David Martin. So the answer should be none. But wait, maybe I missed someone. Let me check again.

Looking at the Tally column for each player again. David Martin's Tally is 1-6.

The other players in Rank 6 are David Martin and the others at rank 9.

But Rank 9's tally is 2-2, 2-2, 2-2, 1-5, 1-5. So no matches there.

So the only player with the same tally as David Martin is himself. Therefore, the answer is None, but maybe the question is phrased as "other players" meaning besides David Martin.

But the question says "what other player had the same tally as david martin?"

So the answer would be none. But wait, maybe the question is looking for all players with the same tally.

But the answer needs to be short, maybe the other player's name.

But since there's only David Martin with 1-6, no others. So the answer is None.</think>

<answer>Answer: None</answer>

HInT's response:

<think>Okay, let's see. The question is asking which other player had the same tally as David Martin. First, I need to find David Martin's tally from the table.

Looking at the table, David Martin is listed under Rank 6. His row shows a Tally of 1-6. So his tally is 1 point and 6 points total. Now I need to check if any other player has the same Tally entry.

Let me go through each row. Starting from the top:

1. Francis Forde: 2-8. Not the same.

2. Niall English: 1-9. No.

3. Kevin Broderick: 3-1. Nope.

3. Gary Kirby (multiple entries): Wait, there are three entries for Gary Kirby here. Let's check their Tally.

The first one is 1-7, then 0-10, and another 0-10. But the second and third entries for Gary Kirby both have Tally 0-10. Hmm, that's different from David Martin's 1-6.

So Gary Kirby's tally is 0-10, which isn't the same as David Martin's 1-6.

Next, looking at Rank 6 again. David Martin's entry: 1-6. Now check others in Rank 6.

The other two players in Rank 6 are Seán McLoughlin and David Martin. Both have Tally 1-6.

So they have the same tally.

Wait, but the question says 'other player' besides David Martin.

So the answer would be Seán McLoughlin. Let me confirm. Yes, Seán McLoughlin also has Tally 1-6.

So he's the other player with the same tally as David Martin.</think>

<answer>Answer: Seán McLoughlin</answer>

Figure 13. Unabridged reasoning comparison for the WikiTQ test example in Figure 5. Top: Qwen2.5-3B (SFT) finds the correct tally but fails to propagate the row-wise constraint, leading to an incorrect answer. Bottom: HInT preserves the row-wise dependency and answers correctly.